

# Assignment 13: Natural Language Processing with Python

Name Ayleen Santander

## Part 1: Text Preprocessing Exercise:

1. Choose a text dataset of your choice and perform text preprocessing using Python. You may use the IMDB dataset attached at the bottom of this page.
2. Implement tokenization, remove stopwords, and perform either stemming or lemmatization.
3. Document each step of the preprocessing process and provide code snippets along with explanations.
4. Discuss any challenges encountered during preprocessing and how you addressed them.

```
#Import Libraries and Necessary Resources
import pandas as pd
import numpy as np
import nltk
import re
import matplotlib.pyplot as plt
import seaborn as sns
!pip install contractions
import contractions
import spacy
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
import matplotlib.pyplot as plt
from textblob import TextBlob
from nltk import pos_tag, word_tokenize
from nltk.corpus import wordnet
from textblob import TextBlob
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.svm import LinearSVC
from sklearn.metrics import confusion_matrix
# Download necessary NLTK resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('stopwords')
```

```
Requirement already satisfied: contractions in /usr/local/lib/python3.13/dist-packages (0.1.73)
Requirement already satisfied: textsearch>=0.0.21 in /usr/local/lib/python3.13/dist-packages (from contractions) (0.0.24)
Requirement already satisfied: anyascii in /usr/local/lib/python3.13/dist-packages (from textsearch>=0.0.21->contractions) (0.3.
Requirement already satisfied: pyahocorasick in /usr/local/lib/python3.13/dist-packages (from textsearch>=0.0.21->contractions)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True
```

```
# Load the dataset/ # Load the dataset into Python using pandas/ I considerate only 1000 rows for this analysis
IMDB = pd.read_csv('IMDB_Dataset.csv', nrows=1000)
print(IMDB.head())
```

```

              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
```

```
# clone data, just considerate review column for this analysis
IMDB_clone = IMDB.copy()
IMDB_clone= IMDB_clone.drop(columns=['sentiment'])

print("Original IMDB columns:", IMDB.columns.tolist())
print("IMDB_clone columns:", IMDB_clone.columns.tolist())
```

```
Original IMDB columns: ['review', 'sentiment']
IMDB_clone columns: ['review']
```

```
# Convert review to lowercase.
IMDB_clone['review'] = IMDB_clone['review'].str.lower()
```

```
#Tokenize the review into words
IMDB_clone['tokens'] = IMDB_clone['review'].apply(word_tokenize)
```

```
#Remove stopwords to filter out common words with little meaning.
stop_words = set(stopwords.words('english'))
IMDB_clone['filtered_tokens'] = IMDB_clone['tokens'].apply(lambda x: [word for word in x if word not in stop_words])
```

```
#Perform stemming to reduce words to their base forms
stemmer = PorterStemmer()
IMDB_clone['stemmed_tokens'] = IMDB_clone['filtered_tokens'].apply(lambda x: [stemmer.stem(word) for word in x])
(IMDB_clone.head())
```

|   | review                                            | tokens                                              | filtered_tokens                                   | stemmed_tokens                                    |
|---|---------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| 0 | one of the other reviewers has mentioned that ... | [one, of, the, other, reviewers, has, mentione...   | [one, reviewers, mentioned, watching, 1, oz, e... | [one, review, mention, watch, 1, oz, episod, '... |
| 1 | a wonderful little production. <br /><br />the... | [a, wonderful, little, production, ., <, br, /, ... | [wonderful, little, production, ., <, br, /, >... | [wonder, littl, product, ., <, br, /, >, <, br... |
| 2 | i thought this was a wonderful way to spend ti... | [i, thought, this, was, a, wonderful, way, to,...   | [thought, wonderful, way, spend, time, hot, su... | [thought, wonder, way, spend, time, hot, summe... |

```
#Perform lemmatizer to reduce words to their base forms or dictionary form (lemma)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN
def clean_and_lemmatize(review):
    # 1. Remove HTML tags first (<br />, <p>, etc.)
    review = re.sub(r'<.*?>', ' ', review)
    # 2. Expand contractions
    review = contractions.fix(review)
    # 3. Lowercase
    review = review.lower()
    # 4. Remove punctuation/numbers, keep only letters and spaces
    review = re.sub(r'^a-z\s', '', review)
    # 5. Tokenize
    tokens = word_tokenize(review)
```

```
# 6. Remove stopwords
tokens = [t for t in tokens if t not in stop_words]
# 7. POS tag + lemmatize
pos_tags = pos_tag(tokens)
lemmatized = [lemmatizer.lemmatize(word, get_wordnet_pos(tag)) for word, tag in pos_tags]
return lemmatized

# Apply pipeline
IMDB_clone['filtered_tokens'] = IMDB_clone['review'].apply(clean_and_lemmatize)
IMDB_clone['text_lemmatized'] = IMDB_clone['filtered_tokens'].apply(lambda tokens: ' '.join(tokens))
IMDB_clone[['review', 'filtered_tokens', 'text_lemmatized']].head(10)
```

|   | review                                            | filtered_tokens                                   | text_lemmatized                                   |
|---|---------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| 0 | one of the other reviewers has mentioned that ... | [one, reviewer, mention, watch, oz, episode, h... | one reviewer mention watch oz episode hook rig... |
| 1 | a wonderful little production. <br /><br />the... | [wonderful, little, production, film, techniqu... | wonderful little production film technique una... |
| 2 | i thought this was a wonderful way to spend ti... | [think, wonderful, way, spend, time, hot, summ... | think wonderful way spend time hot summer week... |
| 3 | basically there's a family where a little boy ... | [basically, family, little, boy, jake, think, ... | basically family little boy jake think zombie ... |
| 4 | petter mattei's "love in the time of money" is... | [petter, matteis, love, time, money, visually,... | petter matteis love time money visually stunni... |
| 5 | probably my all-time favorite movie, a story o... | [probably, alltime, favorite, movie, story, se... | probably alltime favorite movie story selfless... |
| 6 | i sure would like to see a resurrection of a u... | [sure, would, like, see, resurrection, date, s... | sure would like see resurrection date seahunt ... |
| 7 | this show was an amazing, fresh & innovative i... | [show, amazing, fresh, innovative, idea, first... | show amazing fresh innovative idea first air f... |

```
# With Spacy Lemmatizer: Disable pipeline components don't need (only use tagger + lemmatizer)
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])

def clean_text(review):
    review = re.sub(r'<.*?>', ' ', review)
    review = contractions.fix(review)
    review = review.lower()
    return re.sub(r'^a-z\s', '', review)

# Batch process with nlp.pipe() instead of apply() + nlp() per row
cleaned = IMDB['review'].apply(clean_text)
docs = nlp.pipe(cleaned, batch_size=100)

IMDB['filtered_tokens'] = [
    [token.lemma_ for token in doc if token.text not in stop_words and not token.is_space]
    for doc in docs
]
IMDB['text_lemmatized'] = IMDB['filtered_tokens'].apply(lambda tokens: ' '.join(tokens))
IMDB[['review', 'filtered_tokens', 'text_lemmatized']].head(10)
```

|   | review                                            | filtered_tokens                                   | text_lemmatized                                   |
|---|---------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| 0 | One of the other reviewers has mentioned that ... | [one, reviewer, mention, watch, oz, episode, h... | one reviewer mention watch oz episode hook rig... |
| 1 | A wonderful little production. <br /><br />The... | [wonderful, little, production, filming, techn... | wonderful little production filming technique ... |
| 2 | I thought this was a wonderful way to spend ti... | [think, wonderful, way, spend, time, hot, summ... | think wonderful way spend time hot summer week... |
| 3 | Basically there's a family where a little boy ... | [basically, family, little, boy, jake, think, ... | basically family little boy jake think zombie ... |
| 4 | Petter Mattei's "Love in the Time of Money" is... | [petter, matteis, love, time, money, visually,... | petter matteis love time money visually stunni... |
| 5 | Probably my all-time favorite movie, a story o... | [probably, alltime, favorite, movie, story, se... | probably alltime favorite movie story selfless... |
| 6 | I sure would like to see a resurrection of a u... | [sure, would, like, see, resurrection, date, s... | sure would like see resurrection date seahunt ... |
| 7 | This show was an amazing, fresh & innovative i... | [show, amazing, fresh, innovative, idea, first... | show amazing fresh innovative idea first air f... |

## Part 2: Sentiment Analysis Task:

1. Conduct sentiment analysis on a text dataset of customer reviews or social media posts using Python.
2. Preprocess the text data by tokenizing, removing stopwords, and performing either stemming or lemmatization.

3. Implement a sentiment analysis model using a suitable algorithm (e.g., Naive Bayes, Support Vector Machines) and evaluate its performance.
4. Analyze the sentiment distribution of the dataset and identify any insights or trends.
5. Present your findings in a report format, including visualizations and explanations of the sentiment analysis results.

```
# Function to analyze sentiment
def get_sentiment(review):
    return TextBlob(review).sentiment.polarity

IMDB_clone['sentiment_category'] = IMDB_clone['review'].apply(get_sentiment)

# Categorizing sentiment
conditions = [
    (IMDB_clone['sentiment_category'] > 0),
    (IMDB_clone['sentiment_category'] == 0),
    (IMDB_clone['sentiment_category'] < 0)
]
labels = ['Good', 'NO answer', 'Bad']
IMDB_clone['sentiment_category'] = pd.cut(IMDB_clone['sentiment_category'], bins=[-1, 0, 1], labels=['Bad', 'Good'], include_lo

print("Sentiment Analysis Results:")
print(IMDB_clone[['review', 'sentiment_category']].head())
```

Sentiment Analysis Results:

|   | review                                            | sentiment_category |
|---|---------------------------------------------------|--------------------|
| 0 | one of the other reviewers has mentioned that ... | Good               |
| 1 | a wonderful little production. <br /><br />the... | Good               |
| 2 | i thought this was a wonderful way to spend ti... | Good               |
| 3 | basically there's a family where a little boy ... | Bad                |
| 4 | petter mattei's "love in the time of money" is... | Good               |

```
# Function to analyze sentiment
def get_sentiment(review):
    return TextBlob(review).sentiment.polarity

IMDB_clone['sentiment_score'] = IMDB_clone['review'].apply(get_sentiment)

# Categorizing sentiment
conditions = [
    (IMDB_clone['sentiment_score'] > 0),
    (IMDB_clone['sentiment_score'] == 0),
    (IMDB_clone['sentiment_score'] < 0)
]
labels = ['Good', 'No Answer', 'Bad']
IMDB_clone['sentiment_category'] = np.select(conditions, labels, default='Unknown')

("Sentiment Analysis Results:")
(IMDB_clone[['review', 'sentiment_score']])
```

|     | review                                            | sentiment_score |
|-----|---------------------------------------------------|-----------------|
| 0   | one of the other reviewers has mentioned that ... | 0.023433        |
| 1   | a wonderful little production. <br /><br />the... | 0.109722        |
| 2   | i thought this was a wonderful way to spend ti... | 0.354008        |
| 3   | basically there's a family where a little boy ... | -0.057813       |
| 4   | petter mattei's "love in the time of money" is... | 0.217952        |
| ... | ...                                               | ...             |
| 995 | i really love this movie. i remember one time ... | 0.038690        |
| 996 | upon the straight story release in 1999, it wa... | 0.185452        |
| 997 | as a massive fan of the three tv series, i was... | 0.111005        |
| 998 | in the veins of jeepers creepers and the texas... | 0.199306        |
| 999 | now don't get me wrong i love bad movies... no... | -0.169369       |

1000 rows × 2 columns

```
IMDB_clone = IMDB_clone[IMDB_clone['sentiment_category'] != 'NO answer']
(IMDB_clone[['review', 'sentiment_score', 'sentiment_category']])
```

|     | review                                            | sentiment_score | sentiment_category |
|-----|---------------------------------------------------|-----------------|--------------------|
| 0   | one of the other reviewers has mentioned that ... | 0.023433        | Good               |
| 1   | a wonderful little production. <br /><br />the... | 0.109722        | Good               |
| 2   | i thought this was a wonderful way to spend ti... | 0.354008        | Good               |
| 3   | basically there's a family where a little boy ... | -0.057813       | Bad                |
| 4   | petter mattei's "love in the time of money" is... | 0.217952        | Good               |
| ... | ...                                               | ...             | ...                |
| 995 | i really love this movie. i remember one time ... | 0.038690        | Good               |
| 996 | upon the straight story release in 1999, it wa... | 0.185452        | Good               |
| 997 | as a massive fan of the three tv series, i was... | 0.111005        | Good               |
| 998 | in the veins of jeepers creepers and the texas... | 0.199306        | Good               |
| 999 | now don't get me wrong i love bad movies... no... | -0.169369       | Bad                |

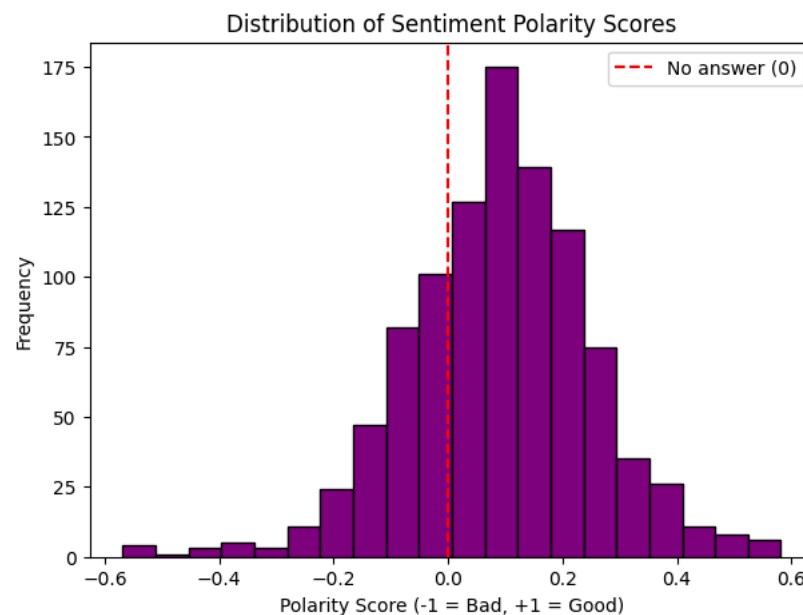
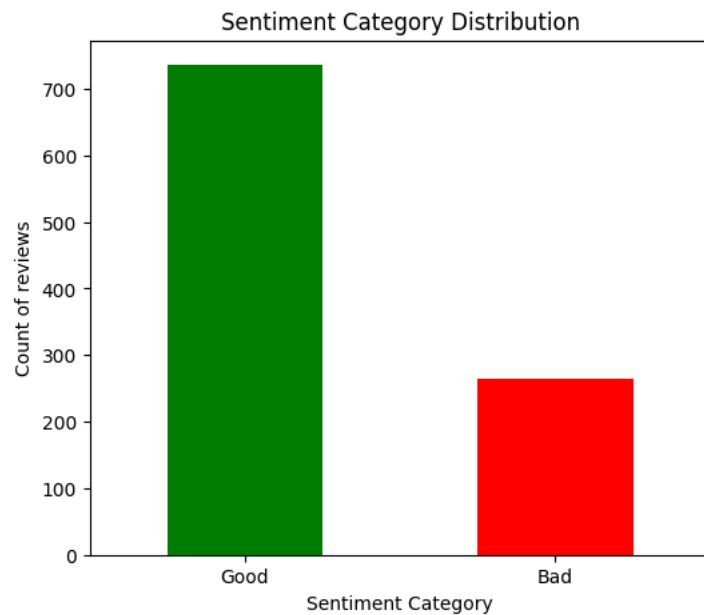
1000 rows × 3 columns

## ▼ Data Visualization and Insights

```
# Function to analyze sentiment
# Plot 1: Bar chart of category counts
color_map = {'Bad': 'Red', 'No Answer': 'blue', 'Good': 'Green'}
counts = IMDB_clone['sentiment_category'].value_counts()
colors = [color_map.get(label, 'gray') for label in counts.index]

plt.figure(figsize=(6, 5))
counts.plot(kind='bar', color=colors)
plt.title('Sentiment Category Distribution')
plt.xlabel('Sentiment Category')
plt.ylabel('Count of reviews')
plt.xticks(rotation=0)
plt.show()

# Plot 2: Histogram of raw polarity scores
plt.figure(figsize=(7, 5))
plt.hist(IMDB_clone['sentiment_score'], bins=20, color='purple', edgecolor='black')
plt.axvline(x=0, color='red', linestyle='--', label='No answer (0)')
plt.title('Distribution of Sentiment Polarity Scores')
plt.xlabel('Polarity Score (-1 = Bad, +1 = Good)')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



```
# Added the Neutral sentiment
neutral_indices = [1, 2 ] # row indices you consider mixed/neutral

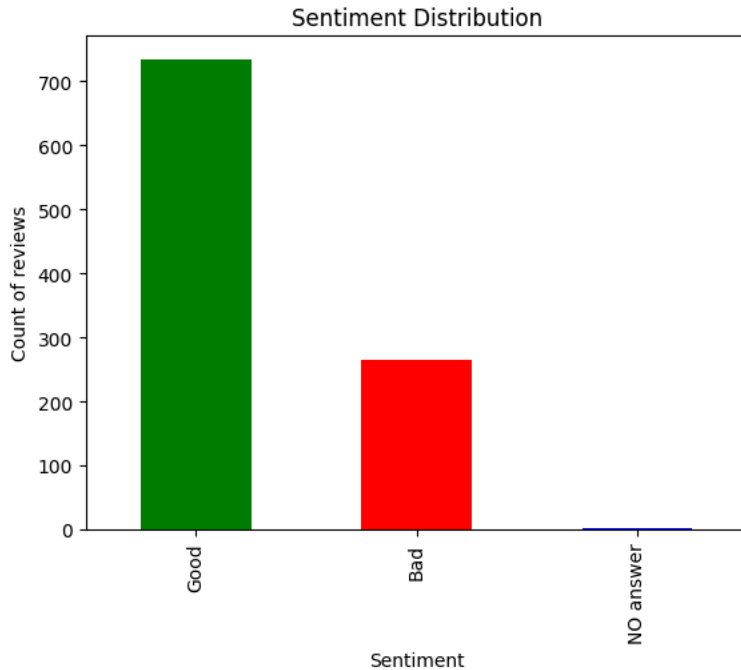
IMDB_clone['sentiment_category'] = IMDB_clone['sentiment_category'].astype(str)
IMDB_clone.loc[neutral_indices, 'sentiment_category'] = 'NO answer'

color_map = {'Bad': 'red', 'NO answer': 'blue', 'Good': 'green'}
counts = IMDB_clone['sentiment_category'].value_counts()
colors = [color_map[label] for label in counts.index]

print(IMDB_clone['sentiment_category'].value_counts())

counts.plot(kind='bar', color=colors)
plt.title('Sentiment Distribution')
plt.xlabel('Sentiment')
plt.ylabel('Count of reviews')
plt.show()
```

```
sentiment_category
Good      734
Bad       264
NO answer    2
Name: count, dtype: int64
```



## Application Models Naive Bayes and SVM Classifiers

```
#Apply training Naive Bayes and SVM Classifiers on text_lemmatized, using sentiment_category (good/bad)
#Step 1: Split data (train/test)
X_train, X_test, y_train, y_test = train_test_split(IMDB_clone['text_lemmatized'], IMDB_clone['sentiment_category'], test_size=
```

```
#Step 2: Vectorize text (TF-IDF)/ Naive and SVM can't operate on raw text, TF-IDF converts each review into a vector
vectorizer = TfidfVectorizer(max_features=5000) #cap vocabulary size to keep it manageable
```

```
#Fit only training data, then transform both - never fit on test data
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
#Step 3: Train Naive Bayes/ is the standard baseline for text classification, its fast, works well with sparse TF-IDF vectors
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_tfidf, y_train)
y_pred_nb = nb_classifier.predict(X_test_tfidf)
```

```
#Step 4: Train SVM, linear SVM tends to outperform Naive Bayes on text classification, finds the optimal separating boundary
#relying on an independence assumption.
svm_classifier = LinearSVC(random_state=42)
svm_classifier.fit(X_train_tfidf, y_train)
y_pred_svm = svm_classifier.predict(X_test_tfidf)
```

```
#Step 5: Evaluate Both Models/ Accuracy, Precision, recall and F1 per class give a fuller picture
print("Naive Bayes Classifier:")
print(classification_report(y_test, y_pred_nb))
print("Accuracy:", accuracy_score(y_test, y_pred_nb))

print('-----SVM-----')
print(classification_report(y_test, y_pred_svm))
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
```

```
Naive Bayes Classifier:
      precision    recall  f1-score   support

    Bad         0.00      0.00      0.00         53
    Good         0.73      1.00      0.85        147
```

```

accuracy      0.73      200
macro avg     0.37      0.50      0.42      200
weighted avg  0.54      0.73      0.62      200

```

Accuracy: 0.735

-----SVM-----

```

              precision    recall  f1-score   support

    Bad         0.74         0.43         0.55         53
    Good         0.82         0.95         0.88        147

```

```

accuracy      0.81      200
macro avg     0.78      0.69      0.71      200
weighted avg  0.80      0.81      0.79      200

```

Accuracy: 0.81

```

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

#Step 6: Visualize Confusion Matrices, comparison Naives Bayes and SVM

```
fig, axes = plt.subplots(1, 2, figsize=(8, 5))
```

```
labels = sentiment_score_labels = ['Bad', 'No answer', 'Good']
```

```

sns.heatmap(confusion_matrix(y_test, y_pred_nb, labels=labels), annot=True, fmt='d', xticklabels=labels, yticklabels=labels, cmap=
axes[0].set_title('Naive Bayes')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

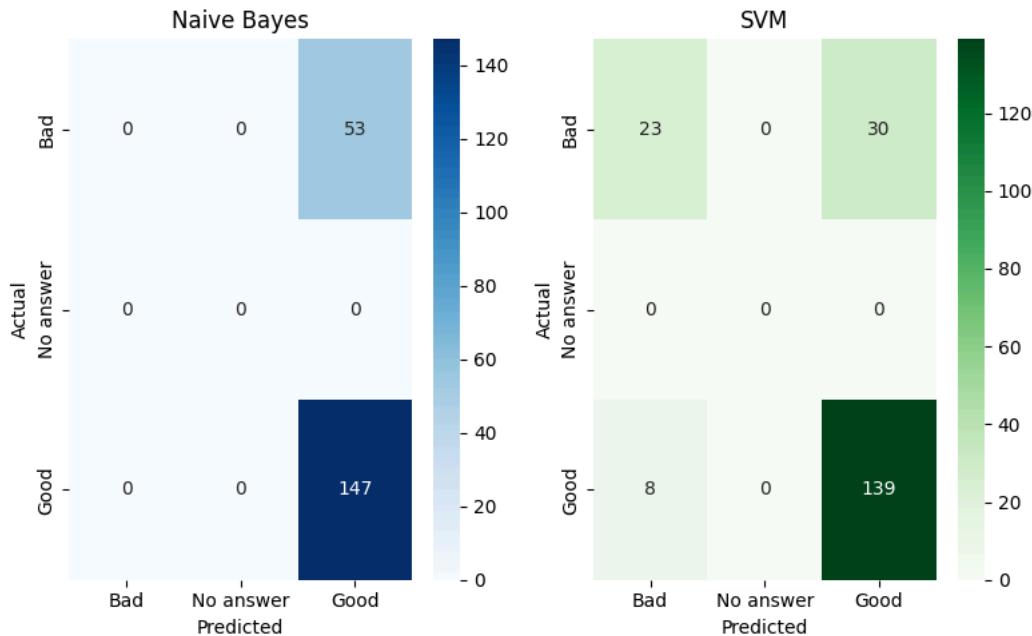
```

```

sns.heatmap(confusion_matrix(y_test, y_pred_svm, labels=labels), annot=True, fmt='d', xticklabels=labels, yticklabels=labels, cm
axes[1].set_title('SVM')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

```

```
plt.tight_layout()
plt.show()
```



#Step 7: Comparison sentimental category original vs preditct

```

comparison = pd.DataFrame({'review': X_test.values, 'actual': y_test.values, 'predicted_nb': y_pred_nb ,
    'predicted_svm': y_pred_svm
})

```

```
comparison.head(10)
```



|   | review                                             | actual | predicted_nb | predicted_svm |  |
|---|----------------------------------------------------|--------|--------------|---------------|--|
| 0 | american nightmare officially tied opinion pat...  | Bad    | Good         | Good          |  |
| 1 | open shot yous marine seriously disrespect you...  | Bad    | Good         | Bad           |  |
| 2 | enjoy watch robert forster main reason rent mo...  | Good   | Good         | Good          |  |
| 3 | karen carpenter story show little singer karen...  | Good   | Good         | Good          |  |
| 4 | see laurel hardy classic film always enjoy com...  | Bad    | Good         | Good          |  |
| 5 | francis ford coppola write direct stunningly p...  | Good   | Good         | Good          |  |
| 6 | okay watched movie twice research heavily net ...  | Good   | Good         | Good          |  |
| 7 | get wrong love bad movie adore bad movie troll...  | Bad    | Good         | Bad           |  |
| 8 | taut organically grip edward dmytryk's crossfir... | Good   | Good         | Good          |  |
| 9 | pepe le moko play charles boyer sort internati...  | Good   | Good         | Good          |  |

Next steps:

[Generate code with comparison](#)[New interactive sheet](#)

```
#Step 8: Correct vs Incorrect predictions
comparison['nb_correct'] = comparison['actual'] == comparison['predicted_nb']
comparison['svm_correct'] = comparison['actual'] == comparison['predicted_svm']

print(f"Naive Bayes accuracy: {comparison['nb_correct'].mean():.3f}")
print(f"SVM accuracy: {comparison['svm_correct'].mean():.3f}")
```

Naive Bayes accuracy: 0.735  
SVM accuracy: 0.810

```
#Step 9: Misclassified / Reviews SVM got wrong
misclassified_svm = comparison[~comparison['svm_correct']]
print(f"{len(misclassified_svm)} misclassified out of {len(comparison)}")
misclassified_svm[['review', 'actual', 'predicted_svm']].head(10)
```

38 misclassified out of 200

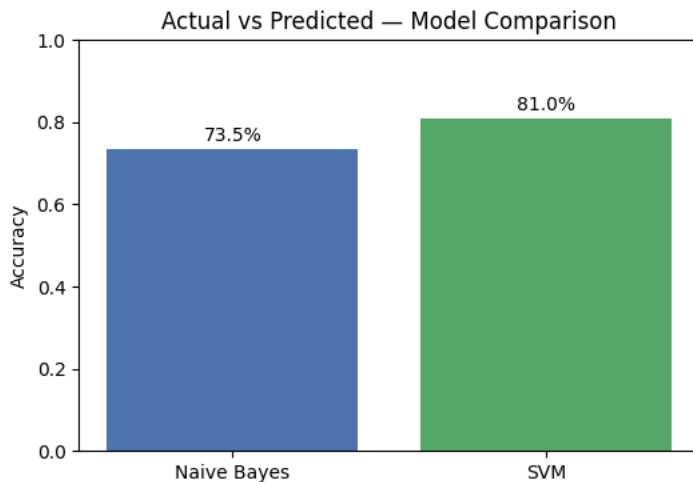
|    | review                                            | actual | predicted_svm |  |
|----|---------------------------------------------------|--------|---------------|--|
| 0  | american nightmare officially tied opinion pat... | Bad    | Good          |  |
| 4  | see laurel hardy classic film always enjoy com... | Bad    | Good          |  |
| 10 | believe ever evil wicked television program ai... | Bad    | Good          |  |
| 11 | mario lewis competitive enterprise institute w... | Bad    | Good          |  |
| 22 | really sad touch movie deal subject child abus... | Bad    | Good          |  |
| 24 | word charles dance character film bollock plot... | Bad    | Good          |  |
| 25 | oh horror see lot gore movie day one make gag ... | Bad    | Good          |  |
| 30 | first realize rating suppose reserve bad bad m... | Good   | Bad           |  |
| 34 | debate movie chose inglorious basterds due am...  | Bad    | Good          |  |
| 35 | besides boring scene oppressive dark movie try... | Good   | Bad           |  |

```
#Step 10: Compare Where Models Disagree With Each Other
disagreements = comparison[comparison['predicted_nb'] != comparison['predicted_svm']]
print(f"Models disagree on {len(disagreements)} reviews")
disagreements[['review', 'actual', 'predicted_nb', 'predicted_svm']].head(10)
```

Models disagree on 31 reviews

|    | review                                            | actual | predicted_nb | predicted_svm |  |
|----|---------------------------------------------------|--------|--------------|---------------|--|
| 1  | open shot yous marine seriously disrespect you... | Bad    | Good         | Bad           |  |
| 7  | get wrong love bad movie adore bad movie troll... | Bad    | Good         | Bad           |  |
| 13 | sorry say film indeed bad reminds cgrade porn ... | Bad    | Good         | Bad           |  |
| 30 | first realize rating suppose reserve bad bad m... | Good   | Good         | Bad           |  |
| 31 | abc gear repertory company another unrealistic... | Bad    | Good         | Bad           |  |
| 35 | besides boring scene oppressive dark movie try... | Good   | Good         | Bad           |  |
| 43 | mighty morphin power ranger get bad television... | Bad    | Good         | Bad           |  |
| 63 | scotty grant cramer would go star great bmovie... | Good   | Good         | Bad           |  |
| 67 | watch first episodes short back felt could tak... | Bad    | Good         | Bad           |  |
| 72 | movie come huge disappointment anime series en... | Bad    | Good         | Bad           |  |

```
#Step 11: Visualize the Comparison
fig, ax = plt.subplots(figsize=(6, 4))
accuracies = [comparison['nb_correct'].mean(), comparison['svm_correct'].mean()]
ax.bar(['Naive Bayes', 'SVM'], accuracies, color=['#4C72B0', '#55A868'])
ax.set_ylim(0, 1)
ax.set_ylabel('Accuracy')
ax.set_title('Actual vs Predicted - Model Comparison')
for i, v in enumerate(accuracies):
    ax.text(i, v + 0.02, f'{v:.1%}', ha='center')
plt.show()
```



## ✓ Sentiment Analysis using VADER

Vader (Valence Aware Dictionary & sentiment Reasoner), took the original text and cleaned punctuation, capitalization and words.

```
import nltk
nltk.download('vader_lexicon')

from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer()

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!
```

```
IMDB_clone['sentiment_score'] = IMDB_clone['review'].apply( sia.polarity_scores)
(IMDB_clone[['review', 'sentiment_score']])
```

|     | review                                            | sentiment_score                                   |
|-----|---------------------------------------------------|---------------------------------------------------|
| 0   | one of the other reviewers has mentioned that ... | {'neg': 0.195, 'neu': 0.749, 'pos': 0.056, 'co... |
| 1   | a wonderful little production. <br /><br />the... | {'neg': 0.053, 'neu': 0.776, 'pos': 0.172, 'co... |
| 2   | i thought this was a wonderful way to spend ti... | {'neg': 0.094, 'neu': 0.714, 'pos': 0.192, 'co... |
| 3   | basically there's a family where a little boy ... | {'neg': 0.138, 'neu': 0.797, 'pos': 0.065, 'co... |
| 4   | petter mattei's "love in the time of money" is... | {'neg': 0.052, 'neu': 0.801, 'pos': 0.147, 'co... |
| ... | ...                                               | ...                                               |
| 995 | i really love this movie. i remember one time ... | {'neg': 0.101, 'neu': 0.811, 'pos': 0.088, 'co... |
| 996 | upon the straight story release in 1999, it wa... | {'neg': 0.108, 'neu': 0.737, 'pos': 0.155, 'co... |
| 997 | as a massive fan of the three tv series, i was... | {'neg': 0.107, 'neu': 0.733, 'pos': 0.16, 'com... |
| 998 | in the veins of jeepers creepers and the texas... | {'neg': 0.181, 'neu': 0.644, 'pos': 0.174, 'co... |
| 999 | now don't get me wrong i love bad movies... no... | {'neg': 0.265, 'neu': 0.554, 'pos': 0.181, 'co... |

1000 rows × 2 columns

```
#Extract individual VADER scores
def get_vader_sentiment(review):
    score = sia.polarity_scores(review)['compound']
    if score >= 0.05:
        return 'good'
    elif score <= -0.05:
        return 'bad'
    else:
        return 'no answer'

IMDB_clone['sentiment_category'] = IMDB_clone['review'].apply(get_vader_sentiment)
print(IMDB_clone['sentiment_category'].value_counts())
```

```
sentiment_category
good          625
bad           371
no answer      4
Name: count, dtype: int64
```

```
#Another way to extract individual VADER Scores/ compound is th single overall sentiment score for the whole piece of text
#compound range (-1 extremely negative), +1 extremely positive
IMDB_clone['bad']= IMDB_clone['sentiment_score'].apply(lambda x: x['neg'])
IMDB_clone['no answer']= IMDB_clone['sentiment_score'].apply(lambda x: x['neu'])
IMDB_clone['good']= IMDB_clone['sentiment_score'].apply(lambda x: x['pos'])
IMDB_clone['compound']= IMDB_clone['sentiment_score'].apply(lambda x: x['compound'])
```

```
(IMDB_clone[['review', 'bad', 'no answer', 'good', 'compound']])
```

|     | review                                            | bad   | no answer | good  | compound |
|-----|---------------------------------------------------|-------|-----------|-------|----------|
| 0   | one of the other reviewers has mentioned that ... | 0.195 | 0.749     | 0.056 | -0.9947  |
| 1   | a wonderful little production. <br /><br />the... | 0.053 | 0.776     | 0.172 | 0.9641   |
| 2   | i thought this was a wonderful way to spend ti... | 0.094 | 0.714     | 0.192 | 0.9605   |
| 3   | basically there's a family where a little boy ... | 0.138 | 0.797     | 0.065 | -0.9213  |
| 4   | petter mattei's "love in the time of money" is... | 0.052 | 0.801     | 0.147 | 0.9744   |
| ... | ...                                               | ...   | ...       | ...   | ...      |
| 995 | i really love this movie. i remember one time ... | 0.101 | 0.811     | 0.088 | -0.0872  |
| 996 | upon the straight story release in 1999, it wa... | 0.108 | 0.737     | 0.155 | 0.9808   |
| 997 | as a massive fan of the three tv series, i was... | 0.107 | 0.733     | 0.160 | 0.9325   |
| 998 | in the veins of jeepers creepers and the texas... | 0.181 | 0.644     | 0.174 | -0.5423  |
| 999 | now don't get me wrong i love bad movies... no... | 0.265 | 0.554     | 0.181 | -0.9867  |

1000 rows × 5 columns

```
#Convert compound score into sentiment labels
def classify_sentiment(compound_score):
```

```

if compound_score >= 0.05:
    return 'good'
elif compound_score <= -0.05:
    return 'bad'
else:
    return 'no answer'
IMDB_clone['sentiment_category'] = IMDB_clone['compound'].apply(classify_sentiment)
print("\nSentiment analysis results:")
print(IMDB_clone[['review', 'good', 'bad', 'compound', 'sentiment_category']].head())

```

Sentiment analysis results:

|   | review                                            | good  | bad   | compound | \ |
|---|---------------------------------------------------|-------|-------|----------|---|
| 0 | one of the other reviewers has mentioned that ... | 0.056 | 0.195 | -0.9947  |   |
| 1 | a wonderful little production. <br /><br />the... | 0.172 | 0.053 | 0.9641   |   |
| 2 | i thought this was a wonderful way to spend ti... | 0.192 | 0.094 | 0.9605   |   |
| 3 | basically there's a family where a little boy ... | 0.065 | 0.138 | -0.9213  |   |
| 4 | petter mattei's "love in the time of money" is... | 0.147 | 0.052 | 0.9744   |   |

|   | sentiment_category |
|---|--------------------|
| 0 | bad                |
| 1 | good               |
| 2 | good               |
| 3 | bad                |
| 4 | good               |

```

#Evaluate sentiment evaluation
print("\n" + "=" * 30)
print("Sentiment Evaluation:")
print("=" * 30)

print ('\nSentiment Counts:')
print(sentiment_counts)

print ('\nSentiment Percentages:')

sentiment_percentages = IMDB_clone['sentiment_category'].value_counts(normalize=True) * 100
print(sentiment_percentages)

```

```

=====
Sentiment Evaluation:
=====

Sentiment Counts:
sentiment_category
good          625
bad           371
no answer         4
Name: count, dtype: int64

Sentiment Percentages:
sentiment_category
good          62.5
bad           37.1
no answer      0.4
Name: proportion, dtype: float64

```

```

#Check average sentiment score
average_compound = IMDB_clone['compound'].mean()
print(f"\nAverage Compound Score: {average_compound:.2f}")

if average_compound >= 0.05:
    print("Overall Sentiment: Good")
elif average_compound <= -0.05:
    print("Overall Sentiment: Bad")
    print("Overall Sentiment: No Answer")

```

Average Compound Score: 0.24  
Overall Sentiment: Good

```

#Identify strongest good and bad records
most_good = IMDB_clone.loc[IMDB_clone['compound'].idxmax()]
most_bad = IMDB_clone.loc[IMDB_clone['compound'].idxmin()]

print("\nMost Good Review:")
print(most_good[['review', 'compound']])

```

```
print('Score:', most_good['compound'])

print("\nMost Bad Review:")
print(most_bad[['review', 'compound']])
print('Score:', most_bad['compound'])
```

```
Most Good Review:
review      i firmly believe that the best oscar ceremony ...
compound                                         0.9996
Name: 410, dtype: object
Score: 0.9996
```

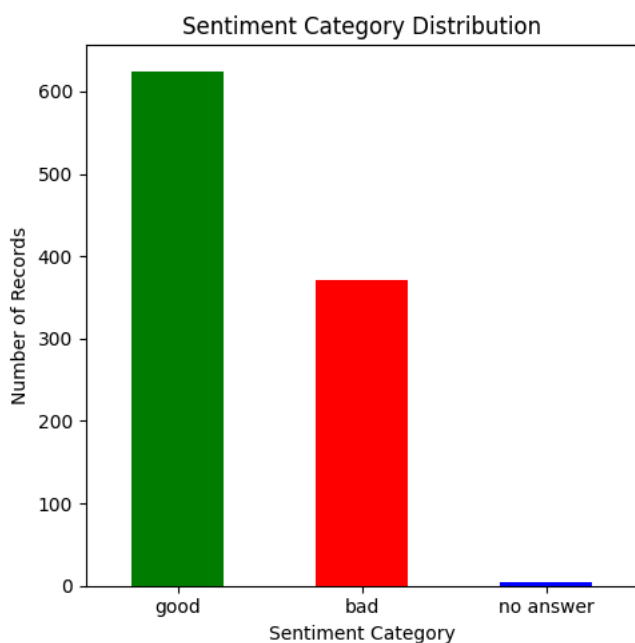
```
Most Bad Review:
review      oh, my goodness. i would have never thought it...
compound                                         -0.9994
Name: 456, dtype: object
Score: -0.9994
```

```
#Sentiment Distribution
print('\nSentiment Distribution')

for sentiment_category, count in sentiment_counts.items():
    percentage = (count / len(IMDB_clone)) * 100
    print(
        f"{sentiment_category}: {count} records"
        f"({percentage:.2f}%)"
    )
```

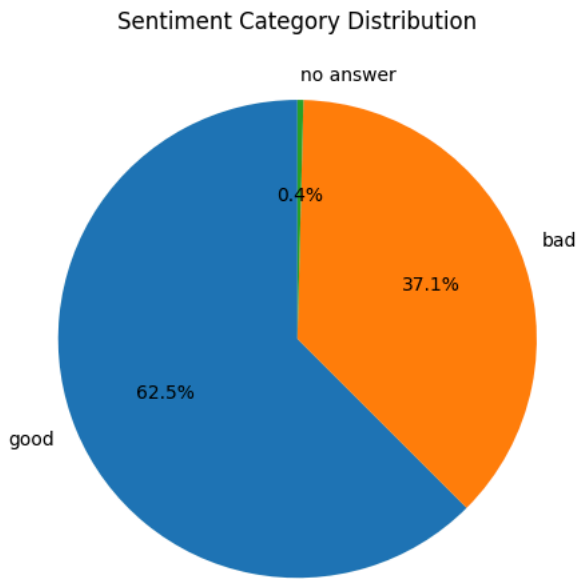
```
Sentiment Distribution
good: 625 records(62.50%)
bad: 371 records(37.10%)
no answer: 4 records(0.40%)
```

```
#Bar Chart
plt.figure(figsize=(5, 5))
sentiment_counts.plot(kind='bar', color=colors)
plt.title('Sentiment Category Distribution')
plt.xlabel('Sentiment Category')
plt.ylabel('Number of Records')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



```
#Pie Chart
plt.figure(figsize=(5, 5))
plt.pie(sentiment_counts.values, labels= sentiment_counts.index, autopct = '%1.1f%', startangle=90)
plt.title('Sentiment Category Distribution')
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))
plt.show()
```



```
#Save Results CSV file
output_columns = [
    'review', 'tokens', 'filtered_tokens', 'stemmed_tokens', 'text_lemmatized', 'sentiment_score',
    'sentiment_category', 'good', 'bad', 'no answer', 'compound'
]
IMDB_clone[output_columns].to_csv('sentiment_analysis_results.csv', index=False)
print("\nResults save as Sentiment_analysis_results.csv")
```

Results save as Sentiment\_analysis\_results.csv

## Application Models Naive Bayes and SVM Classifiers after VADER Technique

```
#Apply training Naive Bayes and SVM Classifieers on text_lemmatized, using sentiment_category (good/bad)#stratify = keeps the
#Step 1: Split data (train/test)
X_train, X_test, y_train, y_test = train_test_split(IMDB_clone['text_lemmatized'], IMDB_clone['sentiment_category'], test_size=
```

```
#Step 2: Vectorize text (TF-IDF)/ Naive and SVM can't operate on raw text, TF-IDF coverts each review into a vector
vectorizer = TfidfVectorizer(max_features = 5000) #cap vocabulary size to keep it manageable
```

```
#Fit only traingin data, then transform both - never fit on test data
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
#Step 3: Train Naive Bayes/ is the standard baseline for text classification, its fast, works well with sparse TF-IDF vectors
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_tfidf, y_train)
y_pred_nb = nb_classifier.predict(X_test_tfidf)
```

```
#Step 4: Train SVM, linaeer SVM tendds to outperform Naive BAYes on text classification, finds the optimal separating boudary t
#relying on an independence assumption.
svm_classifier = LinearSVC(random_state=42)
svm_classifier.fit(X_train_tfidf, y_train)
y_pred_svm = svm_classifier.predict(X_test_tfidf)
```

```
#Step 5: Evaluate Both Models/ Accuracy, Precision, recall and F1 per class give a fuller picture
print("Naive Bayes Classifier:")
print(classification_report(y_test, y_pred_nb))
print("Accuracy:", accuracy_score(y_test, y_pred_nb))

print('-----SVM-----')
```

```
print(classification_report(y_test, y_pred_svm))
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
```

```
Naive Bayes Classifier:
precision    recall  f1-score   support

   bad       1.00     0.03     0.05        74
   good       0.63     1.00     0.77       125
no answer     0.00     0.00     0.00         1

 accuracy          0.64        200
macro avg          0.54     0.34     0.28        200
weighted avg       0.76     0.64     0.50        200
```

Accuracy: 0.635

-----SVM-----

```
precision    recall  f1-score   support

   bad       0.72     0.58     0.64        74
   good       0.77     0.86     0.82       125
no answer     0.00     0.00     0.00         1

 accuracy          0.76        200
macro avg          0.50     0.48     0.49        200
weighted avg       0.75     0.76     0.75        200
```

Accuracy: 0.755

```
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

#Step 6: Visualize Confusion Matrices, comparison Naives Bayes and SVM

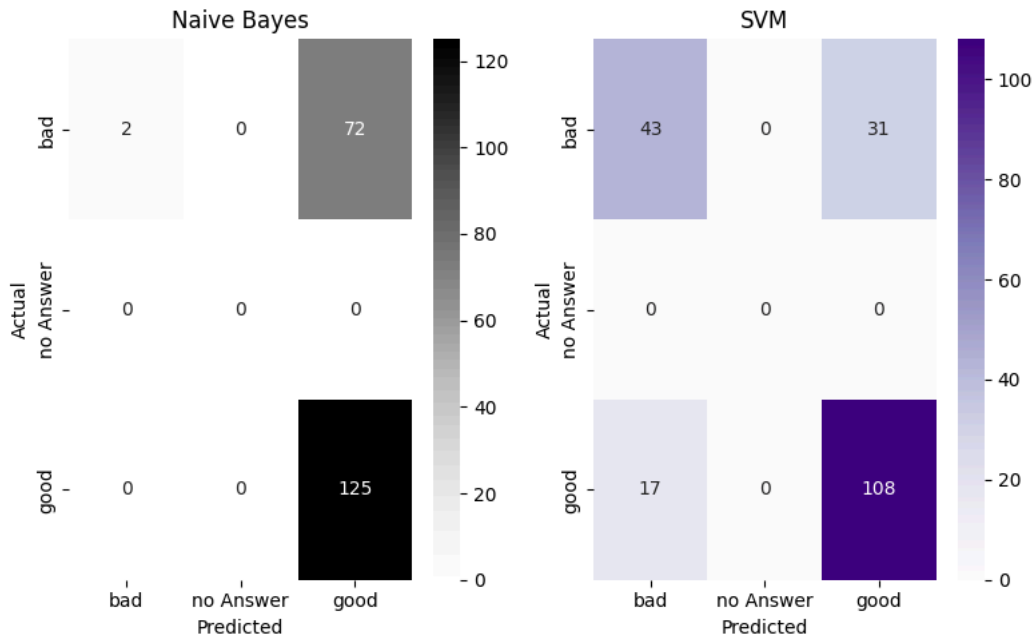
```
fig, axes = plt.subplots(1, 2, figsize=(8, 5))
```

```
labels = sentiment_score_labels = ['bad', 'no Answer', 'good']
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_nb, labels=labels), annot=True, fmt='d',xticklabels=labels, yticklabels=labels, cmap=
axes[0].set_title('Naive Bayes')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_svm, labels=labels), annot=True, fmt='d', xticklabels=labels, yticklabels=labels, cm
axes[1].set_title('SVM')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
```

```
plt.tight_layout()
plt.show()
```



#Step 7: Comparison sentimental category original vs predict

```
comparison = pd.DataFrame({'review': X_test.values, 'actual': y_test.values, 'predicted_nb': y_pred_nb ,
                           'predicted_svm': y_pred_svm
})
```

```
comparison.head(10)
```

|   | review                                            | actual | predicted_nb | predicted_svm |
|---|---------------------------------------------------|--------|--------------|---------------|
| 0 | edmund lowe remind somewhat warren william hea... | good   | good         | good          |
| 1 | grosse pointe blank really quite average film ... | good   | good         | good          |
| 2 | like armand assante cable company summary soun... | bad    | good         | good          |
| 3 | plot michael linnett connors do everything fil... | good   | good         | good          |
| 4 | sometimes patient watch indie horror toetap wa... | bad    | good         | bad           |
| 5 | may go film like assuredly college professor d... | bad    | good         | good          |
| 6 | nothing else tv really bore well watch phat be... | good   | good         | good          |
| 7 | mockumentaries proliferate lately much approac... | bad    | good         | bad           |
| 8 | watch movie get excite see bed scene miss worl... | good   | good         | good          |
| 9 | witty funny intelligent awesome flipping chann... | good   | good         | good          |

Next steps:

[Generate code with comparison](#)

[New interactive sheet](#)

#Step 8: Correct vs Incorrect predictions

```
comparison['nb_correct'] = comparison['actual'] == comparison['predicted_nb']
comparison['svm_correct'] = comparison['actual'] == comparison['predicted_svm']
```

```
print(f"Naive Bayes accuracy: {comparison['nb_correct'].mean():.3f}")
print(f"SVM accuracy: {comparison['svm_correct'].mean():.3f}")
```

Naive Bayes accuracy: 0.635

SVM accuracy: 0.755

#Step 9: Misclassified / Reviews SVM got wrong

```
misclassified_svm = comparison[~comparison['svm_correct']]
print(f"{len(misclassified_svm)} misclassified out of {len(comparison)}")
misclassified_svm[['review', 'actual', 'predicted_svm']].head(10)
```



49 misclassified out of 200

|    | review                                            | actual | predicted_svm |  |
|----|---------------------------------------------------|--------|---------------|--|
| 2  | like armand assante cable company summary soun... | bad    | good          |  |
| 5  | may go film like assuredly college professor d... | bad    | good          |  |
| 13 | saw movie last year medium class say really ha... | bad    | good          |  |
| 15 | open shot yous marine seriously disrespect you... | bad    | good          |  |
| 16 | deny affinity speed movie like fast furious dh... | bad    | good          |  |
| 22 | saw event hit saw documentary take year come a    | bad    | good          |  |